modifies the index in one way or another. For example, we use settings to tweak the number of shards and replicas and to change other index properties. Shards and replicas allow scaling and high availability of data. Mappings define an effective schema for our data for indexing and querying data efficiently. Aliases (alternate names given to indices) let us query across multiple indexes easily and reindex data with zero downtime. We cover all these configurations in detail in the first part of the chapter.

Although it is possible to instantiate indexes manually, it is a tedious, ineffective, and sometimes erroneous process. Instead, organizations should strive for a strategy that develops indexes using index templates. With index templates, we can create indexes with a predefined configuration; and understanding the template mechanism lets us develop indexes for advanced operations such as rollover. Section 6.6 looks at these templates and templating processes in action.

Indexes grow over time along with data, which if unchecked may lead to the system becoming unresponsive. Elasticsearch provides a mechanism to create lifecycle policy definitions to help manage and monitor indexes productively. When an index ages or becomes a certain size, it can be rolled over to a new index, thus preventing unavoidable exceptions.

Similarly, larger indexes created in anticipation of more data can be retired automatically after a set period. Although, index lifecycle management is an advanced topic, it is both fun and engaging. We work through index lifecycle management options in section 6.9. This chapter digs deep into indexing management, index monitoring, and the index lifecycle, so let's jump right in.

NOTE The code for this chapter is available on GitHub (http://mng.bz/Pzyg) and on the book's website (https://www.manning.com/books/elasticsearch-in-action-second-edition).

6.1 Indexing operations

Let's quickly recap what an index is: it is a logical collection of our data backed up by shards (primary and replicas). Documents represented as JSON and having similar attributes (employees, orders, login audit data, news stories by region, and so on) are held in respective indexes. Any index consisting of shards is distributed across various nodes in the cluster. A newly created index is associated with a set number of shards, replicas, and other attributes by default.

We bring indexes to life with custom configurations. We can perform many operations when developing an index, from creating it to closing, shrinking, cloning, freezing, deleting, and other operations. Understanding these operations allows us to set up the system for efficient data storage and search retrieval. Let's begin by looking at creating indexes and the operations involved in instantiating them.

6.2 Creating indexes

When we indexed a document for the first time in earlier chapters, Elasticsearch also created the index *implicitly*. This is one of the ways we can create an index. An